Decomposing structural change in continuous landscapes

surfscape

Contents

1 The problem this package solves **1**

2 Which metrics can see arrangement? **2**

3 Decomposing change **3**

4 How the components scale **4**

5 Field sites **5**

6 Practical notes **5**

1 The problem this package solves

Landscape ecology separates **composition** (how much of what is present) from **configuration** (how it is arranged in space). That distinction is the backbone of patch-based analysis. When landscapes are represented as continuous surfaces instead of patch mosaics, the distinction was never carried across.

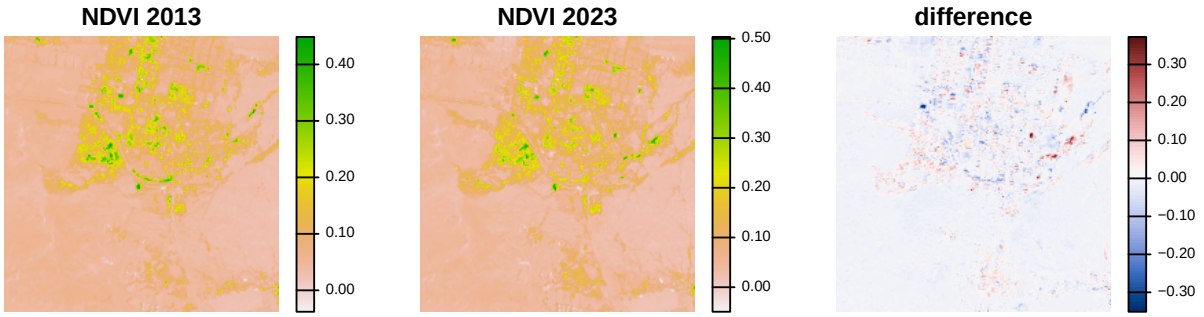
Most of the metrics used on continuous surfaces are functions of the *multiset* of values in the analysis neighbourhood: the amplitude family of surface metrics (`Sa`, `Sq`, `Ssk`, `Sku`, `S10z`), the surface-bearing family derived from the Abbott curve (`Sbi`, `Sci`, `Svi`, `Sk`, `Spk`, `Svk`), and the information-theoretic measures (Shannon entropy, Rao's quadratic entropy). Rearranging every pixel inside the neighbourhood leaves them numerically unchanged. They cannot detect a change in spatial structure, in any landscape, at any scale.

`surfscape` provides two things: a test that establishes this for any metric, and a decomposition that separates the part of an observed change that is due to the value distribution from the part that is due to spatial arrangement.

```
library(terra)
library(surfscape)

z1 <- rast(system.file("extdata", "ndvi_2013.tif", package = "surfscape"))
z2 <- rast(system.file("extdata", "ndvi_2023.tif", package = "surfscape"))
c(mean_2013 = global(z1, "mean", na.rm = TRUE)[1, 1],
  mean_2023 = global(z2, "mean", na.rm = TRUE)[1, 1])
#>  mean_2013  mean_2023
#> 0.06808594 0.06720122
```

```
par(mfrow = c(1, 3), mar = c(1, 1, 2, 3))
plot(z1, axes = FALSE, main = "NDVI 2013")
plot(z2, axes = FALSE, main = "NDVI 2023")
plot(z2 - z1, axes = FALSE, main = "difference",
     col = hcl.colors(100, "Blue-Red 3"))
```



2 Which metrics can see arrangement?

`ss_sensitivity()` samples neighbourhoods, permutes the values inside each one, and compares the observed metric with its permuted values. The permutation preserves the value multiset exactly, so a metric that depends only on that multiset is invariant.

```
s <- ss_sensitivity(z1, w = 15, n = 400, nperm = 49, seed = 1)
s[, c("metric", "family", "arrangement", "csi", "effect")]
#>    metric          family arrangement         csi       effect
#> 22   gcor         texture        TRUE 0.994027725 2.705520e+00
#> 13    sr1 autocorrelation        TRUE 0.990252443 6.365580e+00
#> 15    sdr        gradient        TRUE 0.112355207 5.918229e-01
#> 17   gcon         texture        TRUE 0.109951049 1.569207e+00
#> 18   gdis         texture        TRUE 0.070691787 1.060723e+00
#> 16   slac      lacunarity        TRUE 0.063790640 6.571761e-01
#> 19   ghom         texture        TRUE 0.063164161 8.009612e-01
#> 14    sdq        gradient        TRUE 0.050978538 4.077052e-01
#> 20   gasm         texture        TRUE 0.017226256 1.723748e-01
#> 21   gent         texture        TRUE 0.005617071 1.578899e-01
#> 1   smean       amplitude       FALSE 0.000000000 1.247301e-15
#> 2      sa       amplitude       FALSE 0.000000000 1.330286e-15
#> 3      sq       amplitude       FALSE 0.000000000 6.214624e-16
#> 4     ssk       amplitude       FALSE 0.000000000 7.029213e-16
#> 5     sku       amplitude       FALSE 0.000000000 9.756090e-16
#> 6      sp       amplitude       FALSE 0.000000000 7.608393e-16
#> 7      sv       amplitude       FALSE 0.000000000 7.272860e-16
#> 8      sz       amplitude       FALSE 0.000000000 0.000000e+00
#> 9    s10z       amplitude       FALSE 0.000000000 6.755813e-16
#> 10    scv       amplitude       FALSE 0.000000000 9.157745e-16
#> 11   sent     information       FALSE 0.000000000 1.090125e-15
#> 12   srao     information       FALSE 0.000000000 6.733942e-16
```

Two columns answer two different questions. `arrangement` states whether a metric *can* respond to configuration as a matter of construction. `csi`, the configurational sensitivity index, states how much it *does* respond in this particular landscape: it is `1 - R^2` of the observed metric regressed on its permutation mean, so zero means arrangement contributes nothing.

Metrics from other packages can be audited on exactly the same footing with `ss_sensitivity_fun()`, which takes any function of a neighbourhood matrix.

```
f <- function(m) {
  v <- as.numeric(m)
  d <- v - mean(v)
  c(sd            = sqrt(mean(d^2)),
    skewness      = mean(d^3) / mean(d^2)^1.5,
    shannon       = {p <- table(cut(v, 16)) / length(v); p <- p[p > 0]
                     -sum(p * log(p))},
    mean_sq_diff  = mean((m[, -ncol(m)] - m[, -1])^2))
}
ss_sensitivity_fun(z1, f, w = 15, n = 120, nperm = 19, seed = 1)
#>          metric   n          csi       effect exceed invariant
#> 4 mean_sq_diff 120 1.966342e-01 2.412116e+00      1     FALSE
#> 3      shannon 120 2.220446e-16 0.000000e+00      0      TRUE
#> 1           sd 120 0.000000e+00 0.000000e+00      0      TRUE
#> 2     skewness 120 0.000000e+00 2.284374e-20      0      TRUE
```

The first three are invariant to machine precision; only the last responds to how the values are arranged.

3 Decomposing change

`ss_decompose()` splits the change in a metric between two dates into three additive components:

- **global** – the landscape-wide value distribution shifted;
- **local** – values were redistributed among neighbourhoods;
- **config** – values were rearranged *within* neighbourhoods.

The three sum exactly to the total change. The global component uses a rank transfer between the two dates (`ss_rank_transfer()`), which keeps one date's spatial arrangement and gives it the other date's value distribution; the split is symmetrised over both orderings so that it does not depend on which date is taken first.

```
d <- ss_decompose(z1, z2, metrics = c("sq", "sent", "sr1", "gcon"), w = 15)
names(d)
#>  [1] "sq_total"    "sq_global"   "sq_local"    "sq_config"   "sent_total"
#>  [6] "sent_global" "sent_local"  "sent_config" "sr1_total"   "sr1_global"
#> [11] "sr1_local"   "sr1_config"  "gcon_total"  "gcon_global" "gcon_local"
#> [16] "gcon_config"
```

Additivity is exact, not approximate:

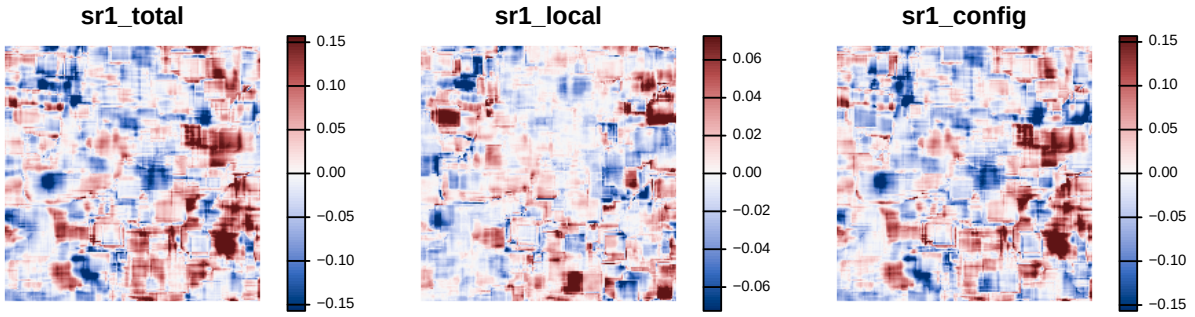
```
v <- values(d[[c("sr1_total", "sr1_global", "sr1_local", "sr1_config")]])
max(abs(v[, 1] - rowSums(v[, 2:4])), na.rm = TRUE)
#> [1] 0
```

For an aspatial metric the configuration component is exactly zero everywhere, which is the point:

```
range(values(d[["sq_config"]]), na.rm = TRUE)
#> [1] -5.551115e-17  5.551115e-17
range(values(d[["sent_config"]]), na.rm = TRUE)
#> [1] 0 0
```

```
par(mfrow = c(1, 3), mar = c(1, 1, 2, 3))
for (k in c("sr1_total", "sr1_local", "sr1_config")) {
  r <- d[[k]]
```

```
  mx <- max(abs(quantile(values(r), c(.02, .98), na.rm = TRUE)))
  plot(clamp(r, -mx, mx, values = TRUE), axes = FALSE, main = k,
       range = c(-mx, mx), col = hcl.colors(100, "Blue-Red 3"))
}
```



4 How the components scale

Composition and configuration do not respond to window size in the same way, so the scale of analysis changes not only the magnitude of a result but which process it is attributed to.

```
sp <- ss_scale_profile(z1, z2, metrics = c("sq", "sr1"),
                       w_seq = c(5, 11, 21, 31))
sp
#>    w metric     n   mad_total  mad_global   mad_local   mad_config share_config
#> 1  5     sq 38416 0.006262127 0.002271056 0.005575243 1.018220e-18 1.625998e-16
#> 2  5    sr1 38416 0.127211332 0.007164621 0.039695853 1.226191e-01 7.569424e-01
#> 3 11     sq 36100 0.006392109 0.002923454 0.005243560 2.691014e-18 4.209900e-16
#> 4 11    sr1 36100 0.062845911 0.006779831 0.022592150 6.153310e-02 7.303515e-01
#> 5 21     sq 32400 0.006407140 0.003455022 0.004743699 6.140439e-18 9.583744e-16
#> 6 21    sr1 32400 0.039067419 0.006091621 0.015530283 3.929967e-02 7.065877e-01
#> 7 31     sq 28900 0.006320343 0.003797954 0.004294036 1.006594e-17 1.592625e-15
#> 8 31    sr1 28900 0.030301777 0.005996177 0.012683431 3.053520e-02 6.838545e-01
#>   r_comp_config r_total_config sign_disagree
#> 1   -0.009697247   -0.009697247     0.7259475
#> 2   -0.050396078    0.945476427     0.1063619
#> 3    0.001674172    0.001674172     0.6148753
#> 4   -0.141278934    0.919679256     0.1268421
#> 5   -0.001744455   -0.001744455     0.5601852
#> 6   -0.254474301    0.895986179     0.1535185
#> 7   -0.003851532   -0.003851532     0.5365052
#> 8   -0.277951576    0.886578811     0.1421453
```

5 Field sites

`ss_extract()` summarises the decomposed change in circular buffers around point locations at several radii, returning a tidy long table ready for multi-scale modelling.

```
set.seed(1)
pts <- data.frame(
  id = 1:6,
  x  = runif(6, xmin(z1) + 600, xmax(z1) - 600),
  y  = runif(6, ymin(z1) + 600, ymax(z1) - 600))
e <- ss_extract(d, pts, radii = c(150, 300, 600))
head(e, 10)
#>    site radius     layer         value
#> 1     1    150  sq_total -0.0019644861
#> 2     2    150  sq_total  0.0023477433
#> 3     3    150  sq_total -0.0030091793
#> 4     4    150  sq_total  0.0024662256
#> 5     5    150  sq_total  0.0007894624
#> 6     6    150  sq_total  0.0025791876
#> 7     1    150 sq_global  0.0044701683
#> 8     2    150 sq_global  0.0057971356
#> 9     3    150 sq_global  0.0023683112
#> 10    4    150 sq_global  0.0027422736
```

6 Practical notes

- Always pass the same `range` to both dates when metrics involve binning (`sent`, `srao`, and the GLCM family). `ss_decompose()` does this for you.
- `csi` is landscape-specific; `arrangement` is not. A metric with `arrangement = TRUE` but a low `csi` is formally able to see configuration but carries little configurational information in the data at hand.
- The decomposition is an algebraic identity, so it is exact for any metric, including metrics supplied by other packages through the rank-transfer route.
- Large rasters are processed block-wise; results are identical to an in-memory pass.

```
sessionInfo()
#> R version 4.3.3 (2024-02-29)
#> Platform: x86_64-pc-linux-gnu (64-bit)
#> Running under: Ubuntu 24.04.4 LTS
#>
#> Matrix products: default
#> BLAS:   /usr/lib/x86_64-linux-gnu/blas/libblas.so.3.12.0
#> LAPACK: /usr/lib/x86_64-linux-gnu/lapack/liblapack.so.3.12.0
#>
#> locale:
#> [1] C
#>
#> time zone: Etc/UTC
#> tzcode source: system (glibc)
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] surfscape_0.2.0 terra_1.7-65
```

```
#>
#> loaded via a namespace (and not attached):
#>  [1] compiler_4.3.3   fastmap_1.1.1    cli_3.6.2        tools_4.3.3
#>  [5] htmltools_0.5.7  yaml_2.3.8       Rcpp_1.0.12      codetools_0.2-19
#>  [9] rmarkdown_2.25   knitr_1.45       xfun_0.41        digest_0.6.34
#> [13] rlang_1.1.3      evaluate_0.23
```